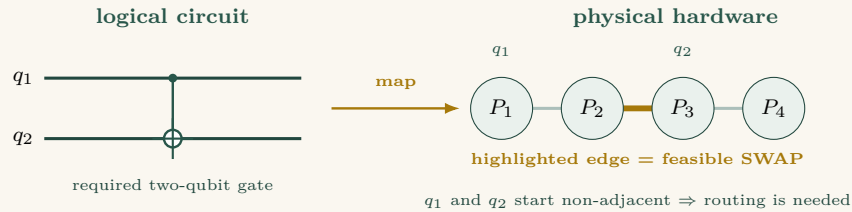




Optimization of Quantum Circuit Routing for Superconducting Qubits

Exact time-expanded routing with calibrated coupling and readout errors



Dimitri
Chrysafis

Marcos Gabriel
Velazquez Ramos

July 31, 2026

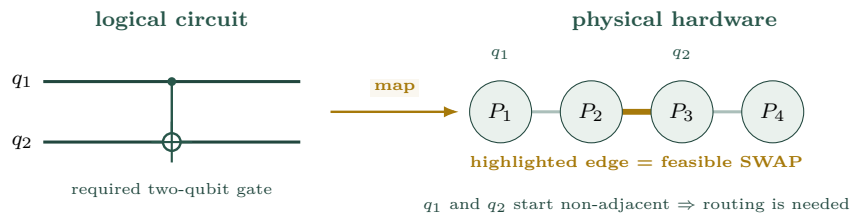
Contents

1	Introduction	3
1.1	Background, history, and motivation	3
1.2	Research questions	3
1.3	Data and assumptions	4
2	Mathematical Model	5
2.1	Sets, parameters, and timeline	5
2.2	Decision variables	5
2.3	Objective function	6
2.4	Constraints and standard form	6
3	Implementation & Solutions	7
3.1	Building and solving one instance	7
3.2	Solution procedure	8
4	Discussion of Results	9
4.1	Topology, layout, and calibration	10
4.2	Circuit structure and scalability	11
4.3	Limitations and sensitivity	12
5	Conclusion	13
A	Reproducibility and Solver Audit	14
B	Julia Implementation	14
	References	28

1 Introduction

This report develops an exact optimization model for routing logical quantum circuits onto a sparse superconducting-qubit architecture. The model chooses legal SWAP operations, certifies that each required two-qubit gate becomes executable, and selects the final measurement sites. We implement the model in Julia with JuMP and HiGHS, then use a structured experiment suite to measure how topology, initial layout, calibration, and circuit structure affect the optimal route.

The routing problem



Quantum circuits are written with *logical* qubits, while a superconducting processor supplies *physical* qubits connected by a hardware-specific graph. A required two-qubit gate is executable only when its logical endpoints occupy adjacent physical sites. If they do not, the compiler must insert SWAP gates. The route changes both the number of extra gates and the physical sites used for final measurement.

1.1 Background, history, and motivation

Quantum routing is a compilation problem created by the gap between the logical circuit written by a programmer and the limited connectivity of a superconducting processor. Hardware-aware transpilation therefore has to choose both an initial layout and a sequence of routing operations. Prior mapping work shows that the physical architecture and calibration quality can materially affect the compiled circuit [3, 4]. This project studies that choice with an exact mixed-integer model whose output can be inspected as a sequence of physical-qubit states.

The problem became especially important as hardware moved from the all-to-all connectivity assumed by abstract circuit diagrams to fixed coupling graphs. Early IBM QX mapping work formalized the need to insert movement operations while respecting a device architecture [4]; later work emphasized that nominally similar qubits and links can have different error rates [3]. The resulting trade-off is both practical and mathematically interesting: the shortest route is not necessarily the lowest-error route.

1.2 Research questions

This project answers three related questions. First, how much does hardware topology change the minimum route error? Second, how much does the initial logical-to-physical layout matter? Third, which interaction patterns increase routing work and MILP size? The experiment design addresses these questions by varying one structural feature at a time around a small baseline whenever the comparison permits.

The model uses the same route-quality idea introduced in the proposal: calibrated connection error for every SWAP plus calibrated readout error at the final physical sites. The experiments use deterministic synthetic calibration profiles, not a claim about one particular backend. This makes the layout and calibration comparisons reproducible while leaving the calibration vector explicit in the Julia model. IBM’s transpilation documentation identifies layout and routing as hardware-aware compilation stages [1], and backend information includes device-specific qubit and gate data [2].

1.3 Data and assumptions

No external backend snapshot is required to reproduce the experiment suite. All hardware graphs, edge errors, readout errors, logical interaction lists, and initial layouts are generated deterministically inside `make_hardware`, `make_circuit`, and `layout_for`. The calibration profiles are synthetic and fixed for reproducibility: they are meant to expose sensitivity of the route to heterogeneous noise, not to claim performance on a dated QPU snapshot. The final submission therefore reports controlled sensitivity results rather than a benchmark of a particular IBM device; replacing these arrays with a dated backend snapshot is reserved for the follow-up study in Section 5.

Several modeling assumptions keep the problem a pure binary MILP. The physical graph is fixed for each solve, edges are undirected, and a required two-qubit gate may be certified in either orientation on a selected edge. The ordered gate list is an input, so the model chooses placements and routing rather than cancelling or reordering gates. Each gate is allotted a fixed budget of $H = 2$ routing transitions, which yields $T = KH$ transitions in total for a circuit with K gates. Within a transition, vertex-disjoint SWAPS may run together, but at most one selected SWAP may touch a given physical site, and movement off the hardware edge set is forbidden. Finally, the objective is an additive comparison score: each selected SWAP on edge e contributes $3\epsilon_e$, and final measurement of a logical qubit on site p contributes ρ_p . That score is useful for ranking routes, but it is not a full-circuit success probability.

The remainder of the report is organized as follows. Section 2 defines the sets, decision variables, objective, and constraints of the time-expanded MILP and presents the model in a compact standard form. Section 3 describes the Julia/JuMP implementation, validates a hand-checkable instance, and records the baseline heavy-7 route. Section 4 discusses the experiment families, interprets the numerical results, and states the main limitations. Section 5 summarizes the findings and proposes one concrete follow-up experiment.

2 Mathematical Model

This section states the routing problem as a binary mixed-integer linear program. We first introduce the sets and parameters, then define the decision variables and objective, and finally collect the constraints into a single standard-form model.

2.1 Sets, parameters, and timeline

Let $\mathcal{P}$ be the set of physical qubit sites and let $\mathcal{E} \subseteq \{\{p, q\} : p, q \in \mathcal{P}, p \neq q\}$ be the undirected hardware edge set. Logical qubits are indexed by $\mathcal{L} = \{1, \dots, L\}$, and the ordered two-qubit gates of the circuit form $\mathcal{K} = \{1, \dots, K\}$. Each gate $k \in \mathcal{K}$ has ordered logical endpoints $a_k, b_k \in \mathcal{L}$. Each edge $e \in \mathcal{E}$ has endpoints $u_e, v_e \in \mathcal{P}$, connection error $\epsilon_e \geq 0$, and incident-edge set $\delta(p) = \{e \in \mathcal{E} : p \in e\}$ for every site $p \in \mathcal{P}$. Site $p \in \mathcal{P}$ also has readout error $\rho_p \geq 0$. The initial layout is encoded by a fixed site $\bar{p}_\ell \in \mathcal{P}$ for each logical qubit $\ell \in \mathcal{L}$.

Routing is time-expanded. With H routing periods allotted to each gate, the horizon is $T = |\mathcal{K}|H$. State times are $\mathcal{T} = \{0, \dots, T\}$ and transition times are $\mathcal{T}^+ = \{1, \dots, T\}$. Gate k is certified at checkpoint $t_k = kH$. Gate orientation indices live in $\mathcal{O} = \{1, 2\}$. Figure 1 shows the timeline for $H = 2$ and two gates. In the Julia implementation, state $t \in \mathcal{T}$ is stored at array index $t + 1$.

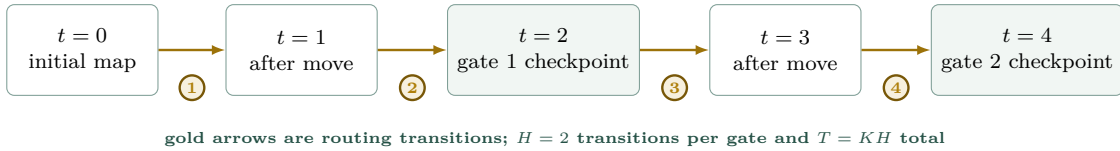


Figure 1: Time-expanded routing timeline with $H = 2$ periods per gate.

2.2 Decision variables

All decision variables are binary. The location variable $x_{\ell pt} \in \{0, 1\}$ equals one when logical qubit $\ell \in \mathcal{L}$ occupies physical site $p \in \mathcal{P}$ at state time $t \in \mathcal{T}$. The move variable $m_{\ell pqt} \in \{0, 1\}$ equals one when logical qubit ℓ moves from site p to site q during transition $t \in \mathcal{T}^+$. The swap variable $s_{et} \in \{0, 1\}$ equals one when hardware edge $e \in \mathcal{E}$ is selected for a SWAP during transition $t \in \mathcal{T}^+$. The gate variable $g_{keo} \in \{0, 1\}$ equals one when ordered gate $k \in \mathcal{K}$ is certified on edge $e \in \mathcal{E}$ in orientation $o \in \mathcal{O}$. Finally, the readout variable $r_{\ell p} \in \{0, 1\}$ equals one when logical qubit ℓ is measured on site p at the end of the horizon.

These five families play complementary roles. The x variables describe the occupancy state; the m variables make every state transition exact; the s variables charge routing cost; the g variables certify that each required interaction is executable at its checkpoint; and the r variables select the final measurement sites that incur readout cost. Disjoint SWAPs may share a transition index, but the model does not assign physical gate durations.

2.3 Objective function

The objective minimizes the sum of routing cost and final readout cost:

$$\min \sum_{t \in \mathcal{T}^+} \sum_{e \in \mathcal{E}} 3\epsilon_e s_{et} + \sum_{\ell \in \mathcal{L}} \sum_{p \in \mathcal{P}} \rho_p r_{\ell p}. \quad (1)$$

The factor of three multiplies each selected edge error because a SWAP is treated as three two-qubit interactions in the additive surrogate. The second double sum charges the calibrated readout error of the final physical site chosen for each logical qubit. Every index that appears in a sum is quantified over an explicitly defined set, so the objective is a well-defined linear function of the binary decision variables.

2.4 Constraints and standard form

The constraints enforce legal occupancy, legal movement, gate certification, and final readout. Every logical qubit occupies exactly one physical site at every state time, and no physical site holds more than one logical qubit. The initial layout is fixed by $x_{\ell p 0} = \mathbb{K}[p = \bar{p}_\ell]$. Move variables link consecutive states, stay-put decisions are incompatible with a SWAP incident to the same site, and off-edge movement is forbidden. A selected SWAP may touch a site at most once per transition and may be charged only if at least one of its endpoints is occupied. Each ordered gate must be certified on exactly one edge and orientation at its checkpoint. Readout assigns each logical qubit to exactly one terminal site consistent with the final occupancy map.

Collecting the objective and constraints yields the following binary MILP, written in a compact standard form that matches the course style of quantifying every index over its set:

$$\begin{aligned} \min \quad & \sum_{t \in \mathcal{T}^+} \sum_{e \in \mathcal{E}} 3\epsilon_e s_{et} + \sum_{\ell \in \mathcal{L}} \sum_{p \in \mathcal{P}} \rho_p r_{\ell p} \\ \text{s.t.} \quad & \sum_{p \in \mathcal{P}} x_{\ell p t} = 1 & \forall \ell \in \mathcal{L}, t \in \mathcal{T}, \\ & \sum_{\ell \in \mathcal{L}} x_{\ell p t} \leq 1 & \forall p \in \mathcal{P}, t \in \mathcal{T}, \\ & x_{\ell p 0} = \mathbb{K}[p = \bar{p}_\ell] & \forall \ell \in \mathcal{L}, p \in \mathcal{P}, \\ & \sum_{q \in \mathcal{P}} m_{\ell p q t} = x_{\ell p, t-1} & \forall \ell \in \mathcal{L}, p \in \mathcal{P}, t \in \mathcal{T}^+, \\ & \sum_{p \in \mathcal{P}} m_{\ell p q t} = x_{\ell q t} & \forall \ell \in \mathcal{L}, q \in \mathcal{P}, t \in \mathcal{T}^+, \\ & m_{\ell p p t} + \sum_{e \in \delta(p)} s_{et} \leq 1 & \forall \ell \in \mathcal{L}, p \in \mathcal{P}, t \in \mathcal{T}^+, \\ & m_{\ell p q t} = 0 & \forall \ell \in \mathcal{L}, p, q \in \mathcal{P}, t \in \mathcal{T}^+ : p \neq q, \{p, q\} \notin \mathcal{E}, \\ & m_{\ell, u_e, v_e, t} \leq s_{et}, \quad m_{\ell, v_e, u_e, t} \leq s_{et} & \forall \ell \in \mathcal{L}, e \in \mathcal{E}, t \in \mathcal{T}^+, \\ & \sum_{e \in \delta(p)} s_{et} \leq 1 & \forall p \in \mathcal{P}, t \in \mathcal{T}^+, \\ & s_{et} \leq \sum_{\ell \in \mathcal{L}} x_{\ell, u_e, t-1} + \sum_{\ell \in \mathcal{L}} x_{\ell, v_e, t-1} & \forall e \in \mathcal{E}, t \in \mathcal{T}^+, \\ & g_{k e 1} \leq x_{a_k, u_e, t_k}, \quad g_{k e 1} \leq x_{b_k, v_e, t_k} & \forall k \in \mathcal{K}, e \in \mathcal{E}, \end{aligned}$$

$$\begin{aligned}
g_{ke2} &\leq x_{a_k, v_e, t_k}, & g_{ke2} &\leq x_{b_k, u_e, t_k} & \forall k \in \mathcal{K}, e \in \mathcal{E}, \\
\sum_{e \in \mathcal{E}} \sum_{o \in \mathcal{O}} g_{keo} &= 1 & \forall k \in \mathcal{K}, \\
\sum_{p \in \mathcal{P}} r_{\ell p} &= 1 & \forall \ell \in \mathcal{L}, \\
r_{\ell p} &\leq x_{\ell p T} & \forall \ell \in \mathcal{L}, p \in \mathcal{P}, \\
\sum_{\ell \in \mathcal{L}} r_{\ell p} &\leq 1 & \forall p \in \mathcal{P}, \\
x_{\ell p t}, m_{\ell p q t}, s_{et}, g_{keo}, r_{\ell p} &\in \{0, 1\} & \text{over the index sets above.}
\end{aligned} \tag{2}$$

If both endpoints of a selected edge are occupied, the two logical qubits exchange sites. If exactly one endpoint is occupied, the occupied qubit may move into the empty site. In either case, the move variables and edge-selection constraints keep the route consistent with the hardware graph.

3 Implementation & Solutions

The model of Section 2 is implemented as a small Julia project. Its core module defines the hardware and circuit types, builds the JuMP model, calls HiGHS, and extracts an inspectable route trace. A separate experiment driver solves the 24 deterministic cases used in this report and writes the CSV tables and figures. Unit tests check the hand-validated line instance, gate feasibility at checkpoints, objective decomposition, and baseline consistency. The PDF is compiled with the provided export script.

The solver wrapper records status, objective, relative gap, model size, SWAP events, gate certificates, and final readout sites, so every numerical claim in the report can be regenerated from code rather than entered by hand. Optimization is run with `set_silent(model)` so that only deliberately printed summaries appear in the validation and experiment workflow below. The complete source for the model, experiment driver, and tests is reproduced in the appendix for independent inspection.

3.1 Building and solving one instance

A single solve has three inputs: a hardware graph, an ordered logical circuit, and an initial logical-to-physical layout. The following call constructs the hand-checkable five-site line, places two logical qubits on nonadjacent sites, and solves the routing MILP with $H = 2$ transitions per gate.

```
include("src/QuantumRouting.jl")
using .QuantumRouting

# Five-site line with uniform edge error 0.010 and readout error 0.012.
hardware = make_hardware(:line5; calibration = :uniform)

# One required interaction between logical qubits 1 and 2.
circuit = Circuit("single_nonlocal", 2, [(1, 2)])

# Place q1 on P1 and q2 on P3 so the pair is initially nonadjacent.
```

```
layout = layout_for(:hand, hardware.n_physical, circuit.n_logical)
```

```
result = solve_routing(hardware, circuit, layout; swaps_per_gate = 2)
```

HiGHS returns status `OPTIMAL`. The extracted solution uses one SWAP, with routing cost 0.0300, readout cost 0.0240, and objective 0.0540. Because the uniform connection error is $\epsilon_e = 0.010$ and the uniform readout error is $\rho_p = 0.012$, the objective identity $3(0.010) + 2(0.012) = 0.0540$ can be checked by hand and matches the solver value exactly.

Inside `build_model`, the JuMP variables mirror the mathematical families of Section 2: `location` implements $x_{\ell pt}$, `swap` implements s_{et} , `move` implements $m_{\ell pqt}$, `gate_use` implements g_{keo} , and `readout` implements $r_{\ell p}$. The objective is coded as the same linear combination that appears in (1), and the constraints are added in the same order as the standard form (2). After `optimize!`, `solve_routing` reads the binary solution, reconstructs the route trace, and returns both the numerical summary and the event lists used by the figures and tables.

3.2 Solution procedure

The implementation follows the same solve–inspect–compare pattern throughout the experiment suite. For each instance, the driver first constructs a hardware graph, an ordered logical circuit, a calibration profile, and an initial layout. It then builds the time-expanded binary MILP with $T = |\mathcal{K}|H$ transitions and the corresponding gate checkpoints. HiGHS solves the model, and the driver rejects any instance that is not certified `OPTIMAL` or that reports a nonzero gap. From each solution, the code extracts the route trace, gate locations, terminal map, readout sites, objective components, model size, and relative gap. Finally, it repeats this process across the topology, layout, calibration, circuit, and scaling families before writing the tables and figures. This separation between model construction and result interpretation means that the same solver record produces both the hand-check validation and the larger comparison tables. The test suite independently checks state injectivity, selected-edge movement, gate certificates at checkpoints, objective decomposition, and terminal readout extraction.

Hand-checkable validation

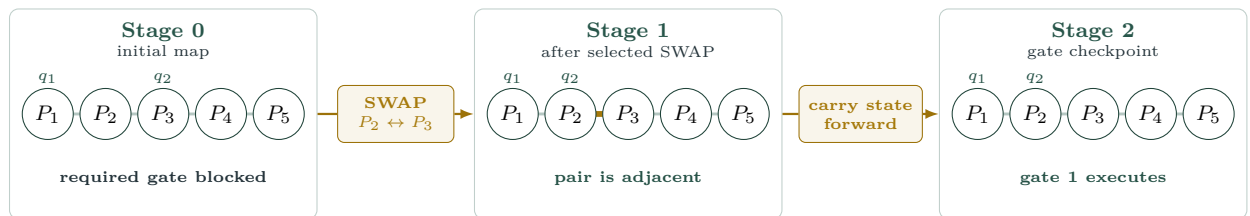
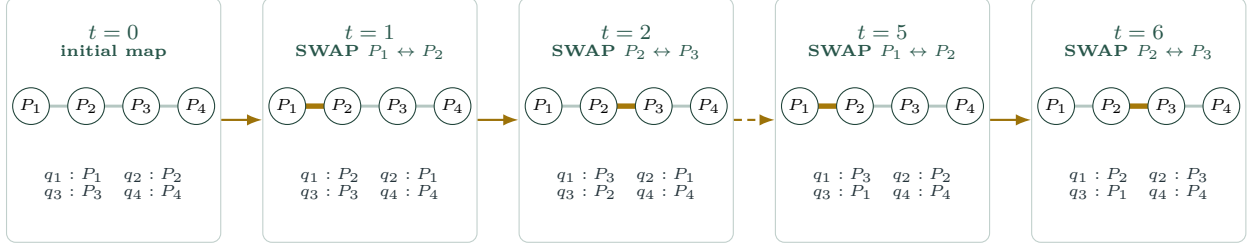


Figure 2: One tie-equivalent hand-check route on the five-site line. The transition moves q_2 from P_3 to P_2 , making the required pair adjacent. With no temporal cost, the same SWAP may instead occur in the second available transition.

Instance	Vars.	Lin. constr.	SWAPs	Route	Readout	Objective	Status
Line-5 validation	156	223	1	0.0300	0.0240	0.0540	OPTIMAL

Baseline route: heavy-7 hardware

State-card abstraction. The cards intentionally show the occupied P_1 – P_4 corridor only. The complete seven-site graph appears in Figure 3; P_5 – P_7 and their incident edges are unused throughout this recorded optimal route.



Trace: P_1 – P_2 at transitions 1 and 5; P_2 – P_3 at 2 and 6. The unshown states $t = 3, 4, 7, 8$ contain no selected SWAP. Gates 1–4 are certified at $t = 2, 4, 6, 8$ on P_3 – P_4 , P_1 – P_2 , P_1 – P_2 , and P_3 – P_4 , respectively. Final map: (P_2, P_3, P_1, P_4) .

4 Discussion of Results

The experiment suite contains 24 deterministic instances, every one of which HiGHS solved to OPTIMAL with a reported relative gap of zero. The baseline anchor is the heavy-7 topology with heterogeneous calibration, the identity initial layout, the cross4 circuit, and $H = 2$ routing periods per gate; that instance has objective 0.1230. Around this anchor we vary one structural feature at a time whenever the comparison permits. Five topologies (line, ring, star, grid, and heavy-7) measure the effect of connectivity. Five initial layouts (identity, reversed, spread, centered, and mixed) measure placement sensitivity on fixed hardware. Four calibration profiles (uniform, heterogeneous, edge-heavy, and readout-heavy) measure the effect of the error vectors. Four interaction patterns (chain3, star4, cross4, and alternating5) and four chain sizes on the 3×3 grid probe circuit structure and model scaling. One additional line-5 hand-check validates the formulation before the larger comparisons.

The suite is deliberately not a full Cartesian product. Layout and calibration families change a single named factor around the baseline. The topology family applies the same heterogeneous-profile construction rule to each graph, so it is an architecture-plus-profile comparison rather than a pure connectivity experiment with identical numerical edge weights. The circuit family varies both interaction pattern and problem size. Because the calibration values are synthetic, the results support controlled method comparisons rather than claims about one dated QPU.

Four headline comparisons set the scale of the effects discussed below. Replacing heavy-7 by grid3x3 reduces the objective from 0.1230 to 0.0540 (a 56.1% drop). Replacing the centered layout by the mixed layout reduces the objective from 0.1550 to 0.0640 (a 58.7% drop). Switching from heterogeneous to readout-heavy calibration raises the objective from 0.1230 to 0.1730 (a 40.7% increase) even though both routes still use four SWAPS. Finally, the local chain3 circuit needs no routing, whereas alternating5 needs five SWAPS, so interaction structure and size jointly drive routing work.

4.1 Topology, layout, and calibration

Hardware topology

Heavy-7 physical connectivity



Figure 3: Synthetic heavy-7 hardware graph.

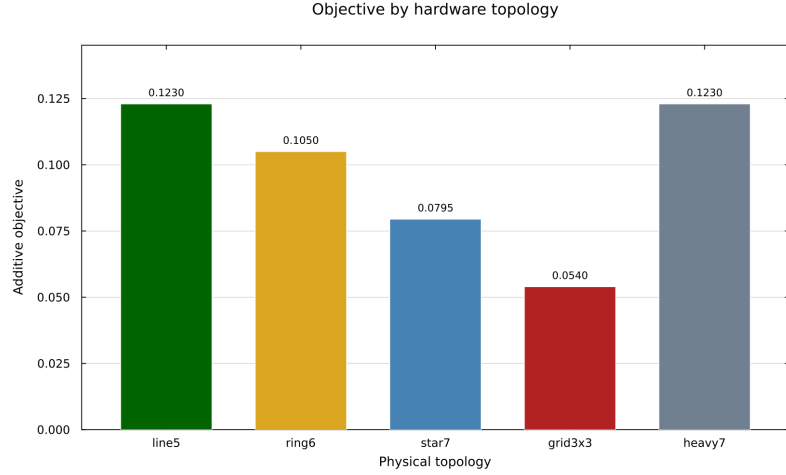


Figure 4: Topology objective for cross4.

Times in the table are one observed HiGHS run.

Topology	SWAPs	Routing	Readout	Objective	Time (s)
line5	4	0.0870	0.0360	0.1230	1.4066
ring6	3	0.0750	0.0300	0.1050	2.1109
star7	2	0.0435	0.0360	0.0795	0.0932
grid3x3	1	0.0180	0.0360	0.0540	0.1815
heavy7	4	0.0870	0.0360	0.1230	2.6123

Under the shared heterogeneous-profile rule, grid3x3 has the lowest combined score and needs only one SWAP. The heavy-7 graph has more sites than the line but the same objective here: qubit count alone is not routability. Because the graph changes the eligible edge and readout arrays, a uniform-calibration topology study would be needed to attribute the full difference to connectivity alone.

Layout and calibration sensitivity

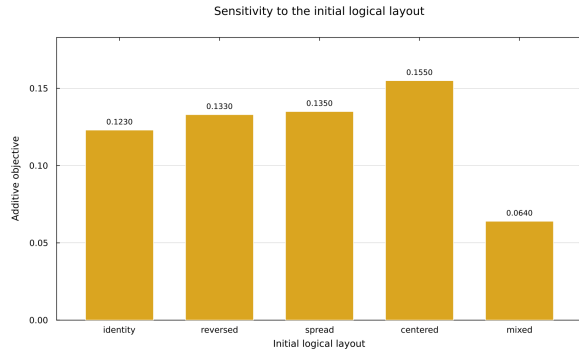


Figure 5: Objective sensitivity to the initial layout on heavy-7.

Layout	SWAPs	Route	Readout	Objective
identity	4	0.0870	0.0360	0.1230
reversed	3	0.0930	0.0400	0.1330
spread	4	0.0990	0.0360	0.1350
centered	4	0.1110	0.0440	0.1550
mixed	2	0.0360	0.0280	0.0640

Layout. Mixed is best. Reversed uses fewer SWAPs than identity but still loses because its selected edges and final readout sites are more expensive.

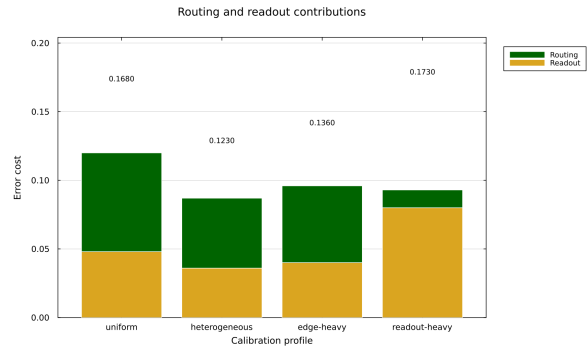


Figure 6: Routing and readout contributions by calibration profile.

Profile	SWAPs	Route	Readout	Objective
uniform	4	0.1200	0.0480	0.1680
heterogeneous	4	0.0870	0.0360	0.1230
edge heavy	4	0.0960	0.0400	0.1360
readout heavy	4	0.0930	0.0800	0.1730

Calibration. Every profile uses four SWAPs, but selected edges, times, and terminal layouts can differ. Readout-heavy data raises the readout term from 0.0360 to 0.0800, more than doubling it relative to heterogeneous data.

4.2 Circuit structure and scalability

Interaction patterns

Pattern	Logical	Gates	SWAPs	Objective
chain3	3	2	0	0.0240
star4	4	3	2	0.0795
cross4	4	4	4	0.1230
alternating5	5	4	5	0.1745

This is a pattern-and-size stress test rather than a one-factor interaction-pattern experiment: logical-qubit and gate counts also vary. Even so, the local chain needs no routing, whereas the cross and alternating sequences repeatedly separate required pairs. A controlled follow-up would hold the logical-qubit and gate counts fixed while varying only the interaction order.

Model-size scaling

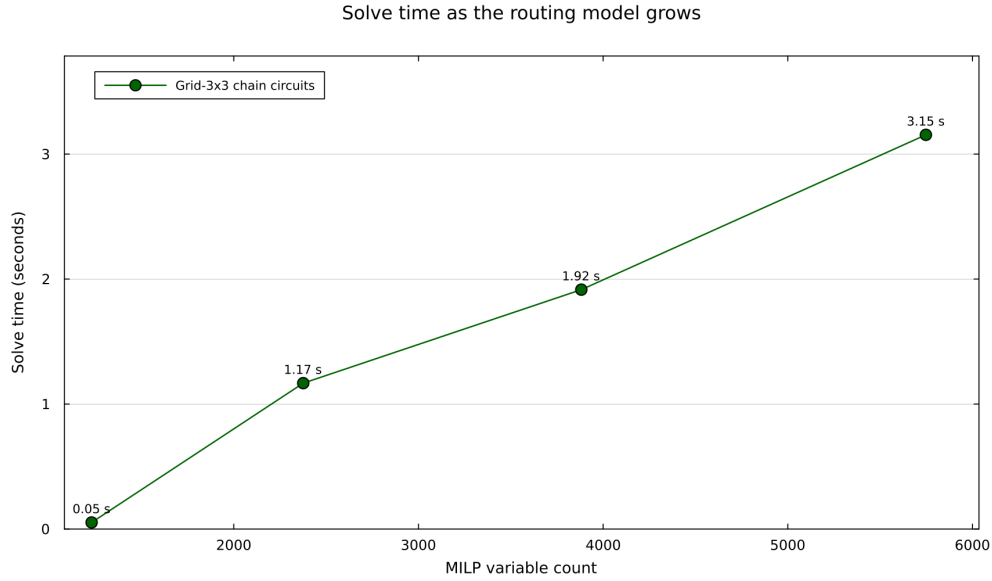


Figure 7: Actual HiGHS solve time versus the model variable count for grid3x3 chain circuits; timings are one observed run.

$$N_{\text{var}} = |\mathcal{L}||\mathcal{P}|(T + 1) + |\mathcal{E}|T + |\mathcal{L}||\mathcal{P}|^2T + 2|\mathcal{K}||\mathcal{E}| + |\mathcal{L}||\mathcal{P}|.$$

Logical	Gates	Variables	Lin. constr.	Objective	Time (s)
3	2	1,230	1,496	0.0240	0.0526
4	3	2,376	2,825	0.0775	1.1672
5	4	3,882	4,554	0.0895	1.9155
6	5	5,748	6,683	0.0995	3.1542

The $|\mathcal{L}||\mathcal{P}|^2T$ movement block is the dominant growth term. All tested cases remain small enough for proven optimality, but the scaling curve identifies the first bottleneck for larger processors.

4.3 Limitations and sensitivity

Like every optimization model, this formulation is useful only inside the assumptions used to build it. The objective is an additive surrogate for route quality rather than a full-circuit failure probability; a natural extension would replace the linear score by a negative-log success probability or by a calibrated noise simulation while keeping the same binary routing decisions. The horizon uses a fixed checkpoint budget $H = 2$ per gate, so the model cannot freely reschedule gates or account for gate duration. Disjoint SWAPS may share a transition index, but the formulation does not enforce a physical resource calendar or crosstalk constraints. Edge errors are undirected scalars, whereas real backends report directional two-qubit errors. Calibration profiles are synthetic, so absolute numerical values should be read as sensitivity results rather than device benchmarks. Finally, the exact MILP is practical only for the small instances studied here: the dominant $|\mathcal{L}||\mathcal{P}|^2T$ movement

block grows quickly, and larger processors will need decomposition, heuristics, or a rolling-horizon restriction of the same model.

The numerical experiments already illustrate sensitivity to the main modeling choices. Topology changes the eligible edge set and therefore the feasible routes. Initial layout changes both the routing term and the final readout term, sometimes in opposite directions, as in the reversed layout that uses fewer SWAPS than identity but still ends with a worse objective. Calibration can change the objective substantially while leaving the SWAP count fixed, which means that counting gates alone is not a sufficient proxy for the calibrated score.

5 Conclusion

This project formulates quantum-circuit routing as a binary time-expanded MILP and solves it with Julia, JuMP, and HiGHS. The formulation prevents occupancy collisions, restricts motion to selected hardware edges, certifies every ordered two-qubit gate at a physical checkpoint, and selects final readout sites under an additive calibrated error score.

All 24 experiment instances solved to proven optimality with zero reported gap. Under the synthetic heterogeneous-profile rule, the grid3x3 topology reduced the objective from the heavy-7 baseline value of 0.1230 to 0.0540, a 56.1% improvement. On fixed heavy-7 hardware, the mixed initial layout reached 0.0640 while the centered layout reached 0.1550, so placement alone can change the optimum by more than a factor of two. Readout-heavy calibration raised the objective to 0.1730 without changing the four-SWAP count, which shows that gate count is an incomplete proxy for the calibrated score. In the circuit-pattern stress test, the local chain needed no routing whereas the alternating pattern needed five SWAPS. Taken together, the experiments show that topology, layout, calibration, and interaction structure are all measurable drivers of route quality on small devices, while the movement-variable block identifies the first scalability bottleneck for larger processors.

One concrete follow-up is immediately doable with the present code. Record a single dated IBM calibration snapshot, replace only the synthetic edge-error and readout-error arrays while keeping the same MILP, and compare the exact route against a standard transpiler on the same 24 cases. Report objective, SWAP count, final readout cost, and solve time for both methods. Because calibration is already isolated from model construction, this experiment tests external validity without redesigning the formulation.

A Reproducibility and Solver Audit

Reproducibility workflow

Run commands

```
julia -project=. -e 'using Pkg; Pkg.instantiate()'
```

```
julia -project=. test/runtests.jl
```

```
julia -project=. scripts/run_experiments.jl
```

```
sh scripts/export_report.sh
```

Environment

Julia	1.12.6
JuMP / HiGHS	1.31.1 / 1.24.1
CSV / DataFrames	0.10.16 / 1.8.2
Plots	1.41.6

The experiment script writes `data/experiment_results.csv`, `tables/`, `figures/`, and the baseline route trace. The PDF is compiled from `report/final_report.tex`.

Solver status across the experiment suite

Case	Logical	Variables	Lin. constr.	Objective	Status
hand_check	2	156	223	0.0540	OPTIMAL
baseline	4	1,960	2,410	0.1230	OPTIMAL
topology_line5	4	1,064	1,390	0.1230	OPTIMAL
topology_ring6	4	1,488	1,880	0.1050	OPTIMAL
topology_star7	4	1,944	2,386	0.0795	OPTIMAL
topology_grid3x3	4	3,144	3,734	0.0540	OPTIMAL
topology_heavy7	4	1,960	2,410	0.1230	OPTIMAL
layout_identity	4	1,960	2,410	0.1230	OPTIMAL
layout_reversed	4	1,960	2,410	0.1330	OPTIMAL
layout_spread	4	1,960	2,410	0.1350	OPTIMAL
layout_centered	4	1,960	2,410	0.1550	OPTIMAL
layout_mixed	4	1,960	2,410	0.0640	OPTIMAL
calibration_uniform	4	1,960	2,410	0.1680	OPTIMAL
calibration_heterogeneous	4	1,960	2,410	0.1230	OPTIMAL
calibration_edge_heavy	4	1,960	2,410	0.1360	OPTIMAL
calibration_readout_heavy	4	1,960	2,410	0.1730	OPTIMAL
circuit_chain3	3	770	972	0.0240	OPTIMAL
circuit_star4	4	1,484	1,827	0.0795	OPTIMAL
circuit_alternating5	5	2,422	2,938	0.1745	OPTIMAL
circuit_cross4	4	1,960	2,410	0.1230	OPTIMAL
scaling_chain3	3	1,230	1,496	0.0240	OPTIMAL
scaling_chain4	4	2,376	2,825	0.0775	OPTIMAL
scaling_chain5	5	3,882	4,554	0.0895	OPTIMAL
scaling_chain6	6	5,748	6,683	0.0995	OPTIMAL

Constraint counts are explicitly added linear rows; the binary-domain declarations are counted in the variable total rather than as additional rows.

B Julia Implementation

Complete Julia source

The complete Julia source used for this report is included in the following appendix: the routing model, experiment driver, and test suite. This makes the PDF independently inspectable even when it is submitted without the repository.

```
1 module QuantumRouting
2
3 # exact time-expanded milp for routing logical two-qubit gates onto a
```

```

4 # sparse superconducting coupling graph the objective is an additive
5 # surrogate for swap connection error plus final readout error
6
7 using HiGHS
8 using JuMP
9
10 # three cnot-equivalent interactions are used as a swap cost multiplier
11 const SWAP_INTERACTION_MULTIPLIER = 3.0
12
13 """physical device: sites, undirected edges, and calibration vectors"""
14 struct Hardware
15     name::String
16     n_physical::Int
17     edges::Vector{Tuple{Int, Int}}
18     connection_errors::Vector{Float64}
19     readout_errors::Vector{Float64}
20     coordinates::Vector{Tuple{Float64, Float64}}
21 end
22
23 """logical circuit as an ordered list of two-qubit interactions"""
24 struct Circuit
25     name::String
26     n_logical::Int
27     gates::Vector{Tuple{Int, Int}}
28 end
29
30 function _connection_errors(profile::Symbol, n_edges::Int, n_physical::Int)
31     if profile == :uniform
32         return fill(0.010, n_edges)
33     elseif profile == :heterogeneous
34         return [0.006 + 0.002 * mod(edge - 1, 4) +
35                 0.0005 * ((edge + n_physical) % 2) for edge in 1:n_edges]
36     elseif profile == :edge_heavy
37         return [0.004 + 0.004 * mod(edge, 3) for edge in 1:n_edges]
38     elseif profile == :readout_heavy
39         return [0.007 + 0.0015 * mod(edge - 1, 3) for edge in 1:n_edges]
40     else
41         error("Unknown connection-error calibration profile: $(profile)")
42     end
43 end
44
45 function _readout_errors(profile::Symbol, n_physical::Int)
46     if profile == :uniform
47         return fill(0.012, n_physical)
48     elseif profile == :heterogeneous
49         return [0.006 + 0.002 * mod(site - 1, 5) for site in 1:n_physical]
50     elseif profile == :edge_heavy
51         return fill(0.010, n_physical)
52     elseif profile == :readout_heavy
53         return [0.014 + 0.004 * mod(site - 1, 4) for site in 1:n_physical]
54     else
55         error("Unknown readout-error calibration profile: $(profile)")
56     end
57 end
58
59 function make_hardware(topology::Symbol; calibration::Symbol = :heterogeneous)
60     if topology == :line5
61         n_physical = 5
62         edges = [(1, 2), (2, 3), (3, 4), (4, 5)]
63         coordinates = [(1.0, 0.0), (2.0, 0.0), (3.0, 0.0), (4.0, 0.0), (5.0, 0.0)]
64     elseif topology == :ring6
65         n_physical = 6
66         edges = [(1, 2), (2, 3), (3, 4), (4, 5), (5, 6), (6, 1)]
67         coordinates = [(cos(2π * (site - 1) / n_physical),
68                        sin(2π * (site - 1) / n_physical)) for site in 1:n_physical]
69     elseif topology == :star7

```

```

70     n_physical = 7
71     edges = [(1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7)]
72     coordinates = [(0.0, 0.0), (-1.0, 1.0), (0.0, 1.35), (1.0, 1.0),
73                   (-1.0, -1.0), (0.0, -1.35), (1.0, -1.0)]
74   elseif topology == :grid3x3
75     n_physical = 9
76     edges = Tuple{Int, Int}[]
77     for row in 1:3, column in 1:3
78       site = 3 * (row - 1) + column
79       column < 3 && push!(edges, (site, site + 1))
80       row < 3 && push!(edges, (site, site + 3))
81     end
82     coordinates = [(Float64(column), Float64(3 - row)) for row in 1:3, column in 1:3]
83     coordinates = vec(coordinates)
84   elseif topology == :heavy7
85     n_physical = 7
86     edges = [(1, 2), (2, 3), (3, 4), (4, 5), (2, 6), (4, 7), (6, 7)]
87     coordinates = [(0.0, 0.0), (1.0, 0.0), (2.0, 0.0), (3.0, 0.0),
88                   (4.0, 0.0), (1.0, -1.0), (3.0, -1.0)]
89   else
90     error("Unknown hardware topology: ${topology}")
91   end
92
93   connection_errors = _connection_errors(calibration, length(edges), n_physical)
94   readout_errors = _readout_errors(calibration, n_physical)
95
96   return Hardware(String(topology), n_physical, edges,
97                  Float64.(connection_errors), Float64.(readout_errors), coordinates)
98 end
99
100 function make_circuit(pattern::Symbol, n_logical::Int)
101   if n_logical < 2
102     error("Circuit requires at least two logical qubits")
103   end
104
105   gates = if pattern == :chain
106     [(logical, logical + 1) for logical in 1:(n_logical - 1)]
107   elseif pattern == :star
108     [(1, logical) for logical in 2:n_logical]
109   elseif pattern == :alternating
110     [(1, n_logical), (2, min(3, n_logical)),
111      (1, min(3, n_logical)), (2, n_logical)]
112   elseif pattern == :cross
113     n_logical == 4 || error("cross circuits require exactly four logical qubits")
114     [(1, 4), (2, 3), (1, 3), (2, 4)]
115   elseif pattern == :nearest
116     [(1, 2), (2, 3), (3, 4), (4, 5)][1:min(4, n_logical - 1)]
117   else
118     error("Unknown circuit pattern: ${pattern}")
119   end
120
121   return Circuit(String(pattern) * string(n_logical), n_logical,
122                 Tuple{Int, Int}.(gates))
123 end
124
125 function layout_for(name::Symbol, n_physical::Int, n_logical::Int)
126   if n_logical > n_physical
127     error("Initial layout cannot use more logical qubits than physical sites")
128   end
129
130   layout = if name == :identity
131     collect(1:n_logical)
132   elseif name == :reversed
133     collect(n_physical:-1:(n_physical - n_logical + 1))
134   elseif name == :spread
135     round.(Int, range(1, n_physical; length = n_logical))

```



```

136     elseif name == :centered
137         first_site = max(1, div(n_physical - n_logical, 2) + 1)
138         collect(first_site:(first_site + n_logical - 1))
139     elseif name == :mixed
140         candidates = [1, n_physical, 2, max(2, n_physical - 1),
141                       3, max(3, n_physical - 2), 4]
142         unique(candidates)[1:n_logical]
143     elseif name == :hand
144         if n_logical != 2 || n_physical < 3
145             error("hand layout requires two logical qubits and at least three physical sites")
146         end
147         [1, 3]
148     else
149         error("Unknown initial layout: $(name)")
150     end
151
152     if length(unique(layout)) != n_logical
153         error("Initial layout is not injective")
154     end
155     return layout
156 end
157
158 function _safe(f, default)
159     try
160         return f()
161     catch
162         return default
163     end
164 end
165
166 function _edge_lookup(hw::Hardware)
167     lookup = zeros{Int, hw.n_physical, hw.n_physical}
168     for (edge_number, (first_site, second_site)) in enumerate(hw.edges)
169         lookup[first_site, second_site] = edge_number
170         lookup[second_site, first_site] = edge_number
171     end
172     return lookup
173 end
174
175 function _incident_edges(hw::Hardware)
176     incident = [Int[] for _ in 1:hw.n_physical]
177     for (edge_number, (first_site, second_site)) in enumerate(hw.edges)
178         push!(incident[first_site], edge_number)
179         push!(incident[second_site], edge_number)
180     end
181     return incident
182 end
183
184 function _validate_hardware(hw::Hardware)
185     n_edges = length(hw.edges)
186     if length(hw.connection_errors) != n_edges ||
187        length(hw.readout_errors) != hw.n_physical ||
188        length(hw.coordinates) != hw.n_physical
189         error("Hardware arrays are inconsistent with the number of sites or edges")
190     end
191     if !all(isfinite, hw.connection_errors) || !all(>=0, hw.connection_errors)
192         error("Connection errors must be finite and nonnegative")
193     end
194     if !all(isfinite, hw.readout_errors) || !all(>=0, hw.readout_errors)
195         error("Readout errors must be finite and nonnegative")
196     end
197
198     seen_edges = Set{Tuple{Int, Int}}{ }
199     for (first_site, second_site) in hw.edges
200         if !(1 <= first_site <= hw.n_physical && 1 <= second_site <= hw.n_physical)
201             error("Hardware edge endpoints are out of range")

```

```

202     end
203     if first_site == second_site
204         error("Hardware edges must connect distinct sites")
205     end
206
207     edge = (min(first_site, second_site), max(first_site, second_site))
208     if edge in seen_edges
209         error("Duplicate undirected hardware edge")
210     end
211     push!(seen_edges, edge)
212 end
213 end
214
215 function _validate_circuit(circuit::Circuit)
216     for (first_logical, second_logical) in circuit.gates
217         if !(1 <= first_logical <= circuit.n_logical &&
218             1 <= second_logical <= circuit.n_logical)
219             error("Circuit gate endpoints are out of range")
220         end
221         if first_logical == second_logical
222             error("Circuit gates must connect distinct logical qubits")
223         end
224     end
225 end
226
227 function _validate_layout(layout::Vector{Int}, n_logical::Int, n_physical::Int)
228     if length(layout) != n_logical || !all(site -> 1 <= site <= n_physical, layout)
229         error("Initial layout length or site indices are invalid")
230     end
231     if length(unique(layout)) != n_logical
232         error("Initial layout is not injective")
233     end
234 end
235
236 """
237     build_model(hw, circuit, initial_layout; swaps_per_gate=2)
238
239 construct the binary time-expanded routing milp each ordered gate is
240 allotted 'swaps_per_gate' transition periods before its checkpoint
241 """
242 function build_model(hw::Hardware, circuit::Circuit, initial_layout::Vector{Int};
243                     swaps_per_gate::Int = 2)
244     _validate_hardware(hw)
245     _validate_circuit(circuit)
246     _validate_layout(initial_layout, circuit.n_logical, hw.n_physical)
247     if swaps_per_gate < 1
248         error("swaps_per_gate must be at least 1")
249     end
250
251     n_logical = circuit.n_logical
252     n_physical = hw.n_physical
253     n_gates = length(circuit.gates)
254     n_edges = length(hw.edges)
255     n_steps = n_gates * swaps_per_gate
256     edge_lookup = _edge_lookup(hw)
257     incident = _incident_edges(hw)
258     edge_u = first.(hw.edges)
259     edge_v = last.(hw.edges)
260
261     model = Model(HiGHS.Optimizer)
262     set_silent(model)
263
264     # decision variables:
265     # location[l,p,t] - logical l occupies physical p at state t
266     # swap[e,t] - edge e is used for a swap at transition t
267     # move[l,p,q,t] - logical l moves from p to q during transition t

```

```

268 # gate_use[k,e,o] - gate k is certified on edge e in orientation o
269 # readout[l,p] - logical l is measured on physical p at the end
270 @variable(model, location[1:n_logical, 1:n_physical, 1:(n_steps + 1)], Bin)
271 @variable(model, swap[1:n_edges, 1:n_steps], Bin)
272 @variable(model, move[1:n_logical, 1:n_physical, 1:n_steps], Bin)
273 @variable(model, gate_use[1:n_gates, 1:n_edges, 1:2], Bin)
274 @variable(model, readout[1:n_logical, 1:n_physical], Bin)
275
276 # initial layout and one-to-one occupancy at every state
277 for logical in 1:n_logical
278     for site in 1:n_physical
279         value = initial_layout[logical] == site ? 1 : 0
280         @constraint(model, location[logical, site, 1] == value)
281     end
282     for time in 1:(n_steps + 1)
283         @constraint(model, sum(location[logical, site, time]
284             for site in 1:n_physical) == 1)
285     end
286 end
287 for site in 1:n_physical, time in 1:(n_steps + 1)
288     @constraint(model, sum(location[logical, site, time]
289         for logical in 1:n_logical) <= 1)
290 end
291
292 # link consecutive states through explicit move variables
293 for logical in 1:n_logical, from in 1:n_physical, time in 1:n_steps
294     @constraint(model, sum(move[logical, from, to, time]
295         for to in 1:n_physical) == location[logical, from, time])
296 end
297 for logical in 1:n_logical, to in 1:n_physical, time in 1:n_steps
298     @constraint(model, sum(move[logical, from, to, time]
299         for from in 1:n_physical) == location[logical, to, time + 1])
300 end
301
302 for logical in 1:n_logical, from in 1:n_physical,
303     to in 1:n_physical, time in 1:n_steps
304     if from == to
305         @constraint(model, move[logical, from, to, time] +
306             sum(swap[edge, time] for edge in incident[from]) <= 1)
307     elseif edge_lookup[from, to] == 0
308         @constraint(model, move[logical, from, to, time] == 0)
309     else
310         edge = edge_lookup[from, to]
311         @constraint(model, move[logical, from, to, time] <= swap[edge, time])
312     end
313 end
314
315 # at most one swap may touch a site empty edges cannot be selected
316 for site in 1:n_physical, time in 1:n_steps
317     @constraint(model, sum(swap[edge, time] for edge in incident[site]) <= 1)
318 end
319 for edge in 1:n_edges, time in 1:n_steps
320     @constraint(model, swap[edge, time] <=
321         sum(location[logical, edge_u[edge], time] for logical in 1:n_logical) +
322         sum(location[logical, edge_v[edge], time] for logical in 1:n_logical))
323 end
324
325 # certify each ordered gate on some hardware edge at its checkpoint
326 for gate in 1:n_gates
327     logical_a, logical_b = circuit.gates[gate]
328     checkpoint = 1 + gate * swaps_per_gate
329     for edge in 1:n_edges
330         @constraint(model, gate_use[gate, edge, 1] <=
331             location[logical_a, edge_u[edge], checkpoint])
332         @constraint(model, gate_use[gate, edge, 1] <=
333             location[logical_b, edge_v[edge], checkpoint])

```

```

334     @constraint(model, gate_use[gate, edge, 2] <=
335         location[logical_a, edge_v[edge], checkpoint])
336     @constraint(model, gate_use[gate, edge, 2] <=
337         location[logical_b, edge_u[edge], checkpoint])
338     end
339     @constraint(model, sum(gate_use[gate, edge, orientation]
340         for edge in 1:n_edges, orientation in 1:2) == 1)
341 end
342
343 # final readout sites are taken from the terminal layout
344 for logical in 1:n_logical
345     @constraint(model, sum(readout[logical, site]
346         for site in 1:n_physical) == 1)
347     for site in 1:n_physical
348         @constraint(model, readout[logical, site] <=
349             location[logical, site, n_steps + 1])
350     end
351 end
352 for site in 1:n_physical
353     @constraint(model, sum(readout[logical, site]
354         for logical in 1:n_logical) <= 1)
355 end
356
357 # minimize 3 * connection error on each swap plus readout error at final sites
358 @objective(model, Min,
359     sum(SWAP_INTERACTION_MULTIPLIER * hw.connection_errors[edge] * swap[edge, time]
360         for edge in 1:n_edges, time in 1:n_steps) +
361     sum(hw.readout_errors[site] * readout[logical, site]
362         for logical in 1:n_logical, site in 1:n_physical))
363
364 variable_count = n_logical * n_physical * (n_steps + 1) +
365     n_edges * n_steps +
366     n_logical * n_physical * n_physical * n_steps +
367     2 * n_gates * n_edges + n_logical * n_physical
368 nonedge_directed = n_physical * (n_physical - 1) - 2 * n_edges
369 constraint_count = n_logical * n_physical +
370     n_logical * (n_steps + 1) +
371     n_physical * (n_steps + 1) +
372     2 * n_logical * n_physical * n_steps
373 constraint_count += n_logical * n_physical * n_steps +
374     n_logical * nonedge_directed * n_steps +
375     n_logical * 2 * n_edges * n_steps
376 constraint_count += n_physical * n_steps + n_edges * n_steps +
377     4 * n_gates * n_edges + n_gates + n_logical +
378     n_logical * n_physical + n_physical
379
380 return (model = model, location = location, swap = swap, move = move,
381     gate_use = gate_use, readout = readout, time_steps = n_steps,
382     variable_count = variable_count, constraint_count = constraint_count,
383     edge_lookup = edge_lookup)
384 end
385
386 function solve_routing(hw::Hardware, circuit::Circuit, initial_layout::Vector{Int};
387     swaps_per_gate::Int = 2)
388     built = build_model(hw, circuit, initial_layout; swaps_per_gate = swaps_per_gate)
389     model = built.model
390
391     memory_before = _safe(() -> Base.gc_live_bytes(), 0)
392     optimize!(model)
393     memory_after = _safe(() -> Base.gc_live_bytes(), 0)
394
395     status = termination_status(model)
396     if !is_solved_and_feasible(model)
397         error("Routing model did not solve to a feasible optimum")
398     end
399 end

```

```

400 n_logical = circuit.n_logical
401 n_physical = hw.n_physical
402 n_gates = length(circuit.gates)
403 n_edges = length(hw.edges)
404 n_steps = built.time_steps
405
406 locations = [findfirst(site -> value(built.location[logical, site, time]) > 0.5,
407                      1:n_physical)
408              for logical in 1:n_logical, time in 1:(n_steps + 1)]
409
410 swap_events = NamedTuple[]
411 for time in 1:n_steps, edge in 1:n_edges
412     if value(built.swap[edge, time]) > 0.5
413         first_site, second_site = hw.edges[edge]
414         cost = SWAP_INTERACTION_MULTIPLIER * hw.connection_errors[edge]
415         push!(swap_events, (transition = time, edge = edge,
416                             u = first_site, v = second_site, cost = cost))
417     end
418 end
419
420 gate_events = NamedTuple[]
421 for gate in 1:n_gates, edge in 1:n_edges, orientation in 1:2
422     if value(built.gate_use[gate, edge, orientation]) > 0.5
423         first_site, second_site = hw.edges[edge]
424         logical_a, logical_b = circuit.gates[gate]
425         push!(gate_events, (gate = gate, logical_a = logical_a,
426                             logical_b = logical_b, edge = edge, u = first_site,
427                             v = second_site, orientation = orientation))
428     end
429 end
430
431 readout_events = NamedTuple[]
432 for logical in 1:n_logical, site in 1:n_physical
433     if value(built.readout[logical, site]) > 0.5
434         push!(readout_events, (logical = logical, physical = site,
435                                cost = hw.readout_errors[site]))
436     end
437 end
438
439 routing_cost = sum(event.cost for event in swap_events; init = 0.0)
440 readout_cost = sum(event.cost for event in readout_events; init = 0.0)
441
442 return (termination_status = string(status), objective = objective_value(model),
443        objective_bound = _safe(() -> objective_bound(model), NaN),
444        relative_gap = _safe(() -> relative_gap(model), NaN),
445        solve_time_sec = _safe(() -> solve_time(model), NaN),
446        variable_count = built.variable_count, constraint_count = built.constraint_count,
447        memory_live_delta_bytes = memory_after - memory_before,
448        locations = locations, swap_events = swap_events,
449        gate_events = gate_events, readout_events = readout_events,
450        routing_cost = routing_cost, readout_cost = readout_cost,
451        model = model, formulation = built)
452 end
453
454 function result_summary(result, hw::Hardware, circuit::Circuit,
455                        layout_name::Symbol, family::Symbol,
456                        calibration::Symbol, topology::Symbol)
457     layout = layout_for(layout_name, hw.n_physical, circuit.n_logical)
458     return (family = String(family), topology = String(topology),
459            calibration = String(calibration), circuit = circuit.name,
460            layout = String(layout_name), n_logical = circuit.n_logical,
461            n_physical = hw.n_physical, n_gates = length(circuit.gates),
462            swap_count = length(result.swap_events),
463            routing_cost = result.routing_cost, readout_cost = result.readout_cost,
464            objective = result.objective, termination_status = result.termination_status,
465            solve_time_sec = result.solve_time_sec,

```

```

466     objective_bound = result.objective_bound, relative_gap = result.relative_gap,
467     variable_count = result.variable_count,
468     constraint_count = result.constraint_count,
469     memory_live_delta_bytes = result.memory_live_delta_bytes,
470     initial_layout = join(layout, ","))
471 end
472
473 export Hardware, Circuit, make_hardware, make_circuit, layout_for,
474     build_model, solve_routing, result_summary
475
476 end

```

Experiment driver source

The driver defines the deterministic experiment cases, solves them, writes the CSV and Markdown tables, and generates the result figures.

```

1  #!/usr/bin/env julia
2
3  ENV["GKSwtstype"] = "100"
4
5  using CSV
6  using DataFrames
7  using Printf
8  using Plots
9
10 include(joinpath(@__DIR__, "..", "src", "QuantumRouting.jl"))
11 using .QuantumRouting
12
13 gr()
14
15 const REPORT_COLORS = [:darkgreen, :goldenrod, :steelblue, :firebrick, :slategray]
16
17 function chart_style(; size = (950, 560))
18     return (
19         size = size,
20         dpi = 180,
21         background_color = :white,
22         foreground_color = :black,
23         framestyle = :box,
24         grid = :y,
25         gridalpha = 0.18,
26         gridlinewidth = 0.4,
27         guidefont = font(10),
28         tickfont = font(9),
29         titlefont = font(12),
30         margin = 7Plots.mm,
31     )
32 end
33
34 function add_bar_labels!(p, values; digits::Int = 3)
35     ymax = maximum(values)
36     ylims!(p, 0, ymax * 1.18)
37     for (i, value) in enumerate(values)
38         annotate!(p, i, value + ymax * 0.035, text(@sprintf("%.6f", digits, value), 8, :black))
39     end
40     return p
41 end
42
43 function fmt(x)
44     if x isa AbstractFloat
45         return @sprintf("%.6f", x)

```

```

46     elseif x === missing
47         return ""
48     else
49         return string(x)
50     end
51 end
52
53 function write_markdown_table(df::DataFrame, path::AbstractString, columns::Vector{Symbol}; digits::Int = 6)
54     open(path, "w") do io
55         headers = [replace(String(c), "_" => " ") for c in columns]
56         println(io, "| " * join(headers, " | ") * " |")
57         println(io, "| " * join(fill("----", length(columns)), " | ") * " |")
58         for row in eachrow(df)
59             values = String[]
60             for c in columns
61                 value = row[c]
62                 if value isa AbstractFloat
63                     push!(values, @sprintf("%.f", digits, value))
64                 else
65                     push!(values, string(value))
66                 end
67             end
68             println(io, "| " * join(values, " | ") * " |")
69         end
70     end
71 end
72
73 function graph_limits(hw::Hardware)
74     xs = first.(hw.coordinates)
75     ys = last.(hw.coordinates)
76     return (minimum(xs) - 0.6, maximum(xs) + 0.6, minimum(ys) - 0.6, maximum(ys) + 0.6)
77 end
78
79 function graph_panel(hw::Hardware; title::String = "", occupied = nothing, highlighted_edges = Int[], subplot
    = nothing)
80     xmin, xmax, ymin, ymax = graph_limits(hw)
81     kw = (xlim = (xmin, xmax), ylim = (ymin, ymax), aspect_ratio = :equal, legend = false,
82         axis = false, grid = false, title = title, size = (500, 420), margin = 3Plots.mm)
83     p = subplot === nothing ? plot(; kw...) : plot(; kw..., subplot = subplot)
84     target = subplot === nothing ? p : p
85     for (e, (u, v)) in enumerate(hw.edges)
86         color = e in highlighted_edges ? :darkgoldenrod2 : :gray60
87         width = e in highlighted_edges ? 4 : 2
88         if subplot === nothing
89             plot!(target, [hw.coordinates[u][1], hw.coordinates[v][1]], [hw.coordinates[u][2], hw.coordinates[
90                 v][2]],
91                 color = color, linewidth = width, label = false)
92         else
93             plot!(target, [hw.coordinates[u][1], hw.coordinates[v][1]], [hw.coordinates[u][2], hw.coordinates[
94                 v][2]],
95                 color = color, linewidth = width, label = false, subplot = subplot)
96         end
97     end
98     xs = first.(hw.coordinates)
99     ys = last.(hw.coordinates)
100     if subplot === nothing
101         scatter!(target, xs, ys, color = :white, markerstrokecolor = :darkgreen, markerstrokewidth = 2,
102             markersize = 12, label = false)
103     else
104         scatter!(target, xs, ys, color = :white, markerstrokecolor = :darkgreen, markerstrokewidth = 2,
105             markersize = 12, label = false, subplot = subplot)
106     end
107     for pidx in 1:hw.n_physical
108         label = "P$pidx"
109         if occupied != nothing
110             logicals = findall(==(pidx), occupied)

```

```

109         if !isempty(logicals)
110             label *= "\nq" * join(logicals, ",q")
111         end
112     end
113     if subplot === nothing
114         annotate!(target, hw.coordinates[pidx][1], hw.coordinates[pidx][2], text(label, 8, :black))
115     else
116         annotate!(target, hw.coordinates[pidx][1], hw.coordinates[pidx][2], text(label, 8, :black),
subplot = subplot)
117     end
118 end
119 return p
120 end
121
122 function case_definitions()
123     return [
124         (case_id = "hand_check", family = :validation, topology = :line5, calibration = :uniform,
125         circuit = Circuit("single_nonlocal", 2, [(1, 2)]), layout = :hand, swaps_per_gate = 2),
126         (case_id = "baseline", family = :baseline, topology = :heavy7, calibration = :heterogeneous,
127         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
128         (case_id = "topology_line5", family = :topology, topology = :line5, calibration = :heterogeneous,
129         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
130         (case_id = "topology_ring6", family = :topology, topology = :ring6, calibration = :heterogeneous,
131         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
132         (case_id = "topology_star7", family = :topology, topology = :star7, calibration = :heterogeneous,
133         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
134         (case_id = "topology_grid3x3", family = :topology, topology = :grid3x3, calibration = :heterogeneous,
135         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
136         (case_id = "topology_heavy7", family = :topology, topology = :heavy7, calibration = :heterogeneous,
137         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
138         (case_id = "layout_identity", family = :layout, topology = :heavy7, calibration = :heterogeneous,
139         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
140         (case_id = "layout_reversed", family = :layout, topology = :heavy7, calibration = :heterogeneous,
141         circuit = make_circuit(:cross, 4), layout = :reversed, swaps_per_gate = 2),
142         (case_id = "layout_spread", family = :layout, topology = :heavy7, calibration = :heterogeneous,
143         circuit = make_circuit(:cross, 4), layout = :spread, swaps_per_gate = 2),
144         (case_id = "layout_centered", family = :layout, topology = :heavy7, calibration = :heterogeneous,
145         circuit = make_circuit(:cross, 4), layout = :centered, swaps_per_gate = 2),
146         (case_id = "layout_mixed", family = :layout, topology = :heavy7, calibration = :heterogeneous,
147         circuit = make_circuit(:cross, 4), layout = :mixed, swaps_per_gate = 2),
148         (case_id = "calibration_uniform", family = :calibration, topology = :heavy7, calibration = :uniform,
149         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
150         (case_id = "calibration_heterogeneous", family = :calibration, topology = :heavy7, calibration = :
heterogeneous,
151         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
152         (case_id = "calibration_edge_heavy", family = :calibration, topology = :heavy7, calibration = :
edge_heavy,
153         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
154         (case_id = "calibration_readout_heavy", family = :calibration, topology = :heavy7, calibration = :
readout_heavy,
155         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
156         (case_id = "circuit_chain3", family = :circuit, topology = :heavy7, calibration = :heterogeneous,
157         circuit = make_circuit(:chain, 3), layout = :identity, swaps_per_gate = 2),
158         (case_id = "circuit_star4", family = :circuit, topology = :heavy7, calibration = :heterogeneous,
159         circuit = make_circuit(:star, 4), layout = :identity, swaps_per_gate = 2),
160         (case_id = "circuit_alternating5", family = :circuit, topology = :heavy7, calibration = :heterogeneous,
161         circuit = make_circuit(:alternating, 5), layout = :identity, swaps_per_gate = 2),
162         (case_id = "circuit_cross4", family = :circuit, topology = :heavy7, calibration = :heterogeneous,
163         circuit = make_circuit(:cross, 4), layout = :identity, swaps_per_gate = 2),
164         (case_id = "scaling_chain3", family = :scaling, topology = :grid3x3, calibration = :heterogeneous,
165         circuit = make_circuit(:chain, 3), layout = :identity, swaps_per_gate = 2),
166         (case_id = "scaling_chain4", family = :scaling, topology = :grid3x3, calibration = :heterogeneous,
167         circuit = make_circuit(:chain, 4), layout = :identity, swaps_per_gate = 2),
168         (case_id = "scaling_chain5", family = :scaling, topology = :grid3x3, calibration = :heterogeneous,
169         circuit = make_circuit(:chain, 5), layout = :identity, swaps_per_gate = 2),

```



```

170     (case_id = "scaling_chain6", family = :scaling, topology = :grid3x3, calibration = :heterogeneous,
171       circuit = make_circuit(:chain, 6), layout = :identity, swaps_per_gate = 2),
172   ]
173 end
174
175 function create_figures(df::DataFrame, root::AbstractString)
176     figure_dir = joinpath(root, "figures")
177     mkpath(figure_dir)
178
179     baseline_hw = make_hardware(:heavy7; calibration = :heterogeneous)
180     p1 = graph_panel(baseline_hw; title = "Heavy-7 physical connectivity")
181     savefig(p1, joinpath(figure_dir, "figure1_hardware_graph.png"))
182
183     top = filter(r -> r.family == "topology", df)
184     top_labels = String.(top.topology)
185     top_x = collect(1:length(top_labels))
186     p3 = bar(top_x, top.objective, color = REPORT_COLORS[1:length(top.topology)],
187       xticks = (top_x, top_labels), xlabel = "Physical topology", ylabel = "Additive objective",
188       title = "Objective by hardware topology", legend = false, bar_width = 0.68,
189       linecolor = :white; chart_style(size = (950, 560))...)
190     add_bar_labels!(p3, top.objective; digits = 4)
191     savefig(p3, joinpath(figure_dir, "figure3_topology_objective.png"))
192
193     lay = filter(r -> r.family == "layout", df)
194     lay_labels = String.(lay.layout)
195     lay_x = collect(1:length(lay_labels))
196     p4 = bar(lay_x, lay.objective, color = REPORT_COLORS[2], xticks = (lay_x, lay_labels),
197       xlabel = "Initial logical layout", ylabel = "Additive objective",
198       title = "Sensitivity to the initial logical layout", legend = false, bar_width = 0.68,
199       linecolor = :white; chart_style(size = (950, 560))...)
200     add_bar_labels!(p4, lay.objective; digits = 4)
201     savefig(p4, joinpath(figure_dir, "figure4_layout_objective.png"))
202
203     cal = filter(r -> r.family == "calibration", df)
204     cal_labels = replace(String.(cal.calibration), "_" => "-")
205     cal_x = collect(1:length(cal_labels))
206     routing = Float64.(cal.routing_cost)
207     readout = Float64.(cal.readout_cost)
208     p5 = bar(cal_x, hcat(routing, readout), labels = ["Routing" "Readout"],
209       xticks = (cal_x, cal_labels), xlabel = "Calibration profile", ylabel = "Error cost",
210       title = "Routing and readout contributions", linecolor = :white,
211       color = [:darkgreen :goldenrod], legend = :outertopright; chart_style(size = (980, 570))...)
212     totals = routing .+ readout
213     ylims!(p5, 0, maximum(totals) * 1.18)
214     for (i, total) in enumerate(totals)
215         annotate!(p5, i, total + maximum(totals) * 0.035, text(sprintf("%.4f", total), 8, :black))
216     end
217     savefig(p5, joinpath(figure_dir, "figure5_calibration_decomposition.png"))
218
219     sc = sort(filter(r -> r.family == "scaling", df), :n_logical)
220     p6 = plot(sc.variable_count, sc.solve_time_sec, marker = :circle, markersize = 6,
221       markerstrokewidth = 1, color = :darkgreen, linewidth = 1.4,
222       xlabel = "MILP variable count", ylabel = "Solve time (seconds)",
223       title = "Solve time as the routing model grows", label = "Grid-3x3 chain circuits",
224       xlims = (minimum(sc.variable_count) * 0.88, maximum(sc.variable_count) * 1.05),
225       ylims = (0, maximum(sc.solve_time_sec) * 1.20); chart_style(size = (980, 570))...)
226     for row in eachrow(sc)
227         annotate!(p6, row.variable_count, row.solve_time_sec + maximum(sc.solve_time_sec) * 0.035,
228           text(sprintf("%.2f s", row.solve_time_sec), 8, :black))
229     end
230     savefig(p6, joinpath(figure_dir, "figure6_scalability.png"))
231
232 end
233
234 function write_tables(df::DataFrame, root::AbstractString)
235     table_dir = joinpath(root, "tables")

```

```

236 mkpath(table_dir)
237
238 solver = select(df, [:case_id, :n_logical, :n_physical, :n_gates, :variable_count, :constraint_count,
239 :termination_status, :objective_bound, :relative_gap, :solve_time_sec, :memory_live_delta_bytes])
240 CSV.write(joinpath(table_dir, "solver_statistics.csv"), solver)
241 write_markdown_table(solver, joinpath(table_dir, "solver_statistics.md"), Symbol.(names(solver)))
242
243 for (family, filename, columns) in [
244     ("topology", "topology_comparison", [:case_id, :topology, :objective, :swap_count, :routing_cost, :
245 readout_cost, :solve_time_sec]),
246     ("layout", "layout_comparison", [:case_id, :layout, :objective, :swap_count, :routing_cost, :
247 readout_cost, :solve_time_sec]),
248     ("calibration", "calibration_comparison", [:case_id, :calibration, :objective, :swap_count, :
249 routing_cost, :readout_cost, :solve_time_sec]),
250     ("circuit", "circuit_comparison", [:case_id, :circuit, :n_logical, :n_gates, :objective, :swap_count, :
251 solve_time_sec]),
252     ("scaling", "scaling_comparison", [:case_id, :n_logical, :n_gates, :variable_count, :constraint_count,
253 :objective, :solve_time_sec]),
254 ]
255     sub = filter(r -> r.family == family, df)
256     selected = select(sub, columns)
257     CSV.write(joinpath(table_dir, filename * ".csv"), selected)
258     write_markdown_table(selected, joinpath(table_dir, filename * ".md"), columns)
259 end
260 end
261
262 function run_experiment_suite(root::AbstractString = normpath(joinpath(@_DIR_, "..")))
263     # define cases and collect solver summaries
264     mkpath(joinpath(root, "data"))
265     rows = NamedTuple[]
266     baseline_result = nothing
267     cases = case_definitions()
268     case_ids = getproperty.(cases, :case_id)
269     if length(unique(case_ids)) != length(case_ids)
270         error("Duplicate case_id in experiment suite")
271     end
272
273     for case in cases
274         # build and solve one deterministic instance
275         hw = make_hardware(case.topology; calibration = case.calibration)
276         layout = layout_for(case.layout, hw.n_physical, case.circuit.n_logical)
277         result = solve_routing(hw, case.circuit, layout; swaps_per_gate = case.swaps_per_gate)
278         if result.termination_status != "OPTIMAL"
279             error("Case $(case.case_id) did not terminate OPTIMAL")
280         end
281         if !isfinite(result.relative_gap) || abs(result.relative_gap) > 1e-9
282             error("Case $(case.case_id) has nonzero relative gap")
283         end
284         if !isapprox(result.objective, result.routing_cost + result.readout_cost;
285             atol = 1e-9, rtol = 1e-9)
286             error("Case $(case.case_id) objective does not match routing + readout costs")
287         end
288         summary = result_summary(result, hw, case.circuit, case.layout, case.family, case.calibration, case.
289 topology)
290         push!(rows, merge((case_id = case.case_id,), summary))
291         if case.case_id == "baseline"
292             baseline_result = result
293         end
294         println(sprintf("%-28s objective=%0.6f swaps=%d time=%0.4fs status=%s", case.case_id,
295             result.objective, length(result.swap_events), result.solve_time_sec, result.termination_status))
296     end
297
298     df = DataFrame(rows)
299     # write tables, figures, and the baseline trace
300     CSV.write(joinpath(root, "data", "experiment_results.csv"), df)
301     write_tables(df, root)

```

```

296 create_figures(df, root)
297
298 open(joinpath(root, "data", "baseline_route.txt"), "w") do io
299     println(io, "Baseline hardware: heavy7")
300     println(io, "Baseline circuit: cross4")
301     println(io, "Initial logical-to-physical layout: ", layout_for(::identity, 7, 4))
302     println(io, "Final logical locations: ", baseline_result.locations[:, end])
303     println(io, "SWAP events:")
304     for event in baseline_result.swap_events
305         println(io, "  transition $(event.transition): P$(event.u) <-> P$(event.v), cost=$(fmt(event.cost))")
306     end
307     println(io, "Gate locations:")
308     for event in baseline_result.gate_events
309         println(io, "  gate $(event.gate) (q$(event.logical_a),q$(event.logical_b)) on P$(event.u)-P$(event.v)")
310     end
311     println(io, "Readout events:")
312     for event in baseline_result.readout_events
313         println(io, "  q$(event.logical) on P$(event.physical), cost=$(fmt(event.cost))")
314     end
315 end
316
317 return df
318 end
319
320 if abspath(PROGRAM_FILE) == @__FILE__
321     run_experiment_suite()
322 end

```

Test suite source

These tests verify state injectivity, selected-edge movement, gate feasibility at checkpoints, objective decomposition, and baseline readout extraction.

```

1 using Test
2
3 include(joinpath(@__DIR__, "..", "src", "QuantumRouting.jl"))
4 using .QuantumRouting
5
6 function edge_exists(hw, first_site, second_site)
7     return any((u == first_site && v == second_site) ||
8               (u == second_site && v == first_site)
9               for (u, v) in hw.edges)
10 end
11
12 function assert_route_trace_is_consistent(result, hw, circuit; swaps_per_gate::Int)
13     for t in axes(result.locations, 2)
14         @test length(unique(result.locations[:, t])) == size(result.locations, 1)
15     end
16
17     for t in 1:(size(result.locations, 2) - 1)
18         selected_edges = Set{Tuple{Int, Int}}((min(event.u, event.v), max(event.u, event.v))
19         for event in result.swap_events if event.transition == t)
20         for logical in axes(result.locations, 1)
21             from_site = result.locations[logical, t]
22             to_site = result.locations[logical, t + 1]
23             if from_site != to_site
24                 @test edge_exists(hw, from_site, to_site)
25                 edge = (min(from_site, to_site), max(from_site, to_site))
26                 @test edge in selected_edges
27             end
28         end
29     end
30 end

```

```

28     end
29 end
30
31 for event in result.gate_events
32     checkpoint = 1 + event.gate * swaps_per_gate
33     observed = Set((result.locations[event.logical_a, checkpoint],
34                    result.locations[event.logical_b, checkpoint]))
35     certified = Set((event.u, event.v))
36     @test observed == certified
37     @test edge_exists(hw, event.u, event.v)
38 end
39
40 for event in result.readout_events
41     @test event.physical == result.locations[event.logical, end]
42 end
43 end
44
45 @testset "Quantum routing model" begin
46     @test_throws Exception make_hardware(:unknown)
47     @test_throws Exception make_circuit(:cross, 3)
48     @test_throws Exception layout_for(:hand, 2, 2)
49
50     hw = make_hardware(:line5; calibration = :uniform)
51     circuit = Circuit("hand", 2, [(1, 2)])
52     layout = layout_for(:hand, hw.n_physical, circuit.n_logical)
53     @test_throws Exception build_model(hw, Circuit("bad_gate", 2, [(1, 3)]), layout)
54     @test_throws Exception build_model(hw, Circuit("self_gate", 2, [(1, 1)]), layout)
55     bad_calibration = Hardware("bad", hw.n_physical, hw.edges, hw.connection_errors[1:end-1], hw.
56                               readout_errors, hw.coordinates)
56     @test_throws Exception build_model(bad_calibration, circuit, layout)
57     result = solve_routing(hw, circuit, layout; swaps_per_gate = 2)
58
59     @test result.termination_status == "OPTIMAL"
60     @test length(result.swap_events) == 1
61     @test result.routing_cost > 0
62     @test result.readout_cost > 0
63     @test result.objective ≈ result.routing_cost + result.readout_cost atol = 1e-8
64     @test abs(result.locations[1, 3] - result.locations[2, 3]) == 1
65     @test sort(result.locations[:, 3]) in ([1, 2], [2, 3])
66     assert_route_trace_is_consistent(result, hw, circuit; swaps_per_gate = 2)
67
68     heavy = make_hardware(:heavy7; calibration = :heterogeneous)
69     cross = make_circuit(:cross, 4)
70     baseline = solve_routing(heavy, cross, layout_for(:identity, 7, 4); swaps_per_gate = 2)
71     @test baseline.termination_status == "OPTIMAL"
72     @test length(baseline.gate_events) == length(cross.gates)
73     @test length(baseline.readout_events) == cross.n_logical
74     assert_route_trace_is_consistent(baseline, heavy, cross; swaps_per_gate = 2)
75 end

```

References

- [1] IBM Quantum, “Introduction to transpilation,” IBM Quantum Documentation, 2026. quantum.cloud.ibm.com/docs/en/guides/transpile.
- [2] IBM Quantum, “View backend details,” IBM Quantum Documentation, 2026. quantum.cloud.ibm.com/docs/en/guides/qpu-information.
- [3] S. S. Tannu and M. K. Qureshi, “Not all qubits are created equal: A case for variability-aware policies for NISQ-era quantum computers,” *Proceedings of ASPLOS*, 2019. doi:10.1145/3297858.3304007.
- [4] A. Zulehner, A. Paler, and R. Wille, “An efficient methodology for mapping quantum circuits to the IBM QX architectures,” *IEEE Transactions on Computer-Aided Design*, 38(7), 2019, 1226–1236. doi:10.1109/TCAD.2018.2846658.
- [5] JuMP developers, “JuMP documentation,” 2026. jump.dev/JuMP.jl/stable.
- [6] HiGHS developers, “HiGHS documentation,” 2026. ergo-code.github.io/HiGHS/dev.